

Programming Languages

Lecture 10

Programming Languages

- Historical Perspective
- Traditional Programming Concepts
- Procedural Units
- Language Implementation
- Object Oriented Programming
- Programming Concurrent Activities
- Declarative Programming

Programming Language

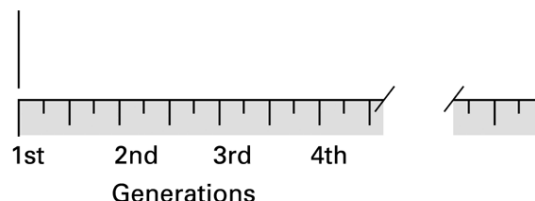
- Programming language is a language that can “precisely describe” an algorithm to a computer so that it can execute it.
-
- ❑ There are two extremes in designing a PL:
 - Use human language
 - Use machine code
 - ❑ Human languages as programming languages
 - Imprecise
 - Inefficient (for computer as well as human)
 - Easy to use
 - Hard to debug
 - ❑ Machine instruction code as programming languages
 - Precise
 - Efficient for computers
 - Verbose to use
 - Hard to debug

Generations of programming languages

- **1st Generation: Machine Language**
 - Machine-dependent, very hard to read, verbose and fast
- **2nd Generation: Assembly Language**
 - Machine-dependent, uses mnemonic names to reference op-codes and memory addresses, verbose and fast
- **3rd Generation: FORTRAN, COBOL, C, C++, LISP**
 - Machine-independent, operations are expressed at higher level, less verbose, easier to read, but slower.
- **4th Generation: Java, C#, Python**
 - Machine-independent, platform-independent, abstractions are allowed, easy to read, slow to execute.

Problems solved in an environment
in which the human must conform
to the machine's characteristics

Problems solved in an environment
in which the machine conforms
to the human's characteristics



Assembly Language

- Since machine codes are too hard to remember (in real-life machine language may contain hundreds of different instructions), each CPU vendor designs an “easy-to-remember” names for each op-code.
- Assembly language - a mnemonic system for representing machine instructions
 - Mnemonic names for op-codes
 - Reserved names for all registers
 - Program **variables** or **identifiers**: Descriptive names for memory locations, chosen by the programmer
- Characteristics:
 - One-to-one correspondence between machine instructions and assembly instructions
 - Programmers are forced to think like a machine
 - Inherently machine-dependent
 - **Assembler** is a special program which translates assembly code to machine code.

Assembly Language Example

Machine language program

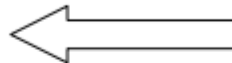
```
156C
166D
5056
30CE
C000
```

Assembly language program

```
LD    R5, [Price]
LD    R6, [ShippingCharge]
ADDI  R0, R5 R6
ST    R0, [TotalCost]
HLT

;
ORG   6Ch
Price                db 25
ShippingCharge       db 5
TotalCost            db 00
```

assembler



Definition of mnemonics:

LD	means "load"
ADDI	means "Integer addition"
ST	means "store"
HLT	means "halt"
ORG	means "origin"
db	means "define byte"

6/40

Third Generation Language

- Uses high-level primitives
 - Similar to our pseudocode in previous lesson
- Machine independent (mostly)
- Examples:
 - FORTRAN – developed for numerical computations in science
 - COBOL – developed for business applications
- Each primitive corresponds to a sequence of machine language instructions
- Converted to machine language by a program called a **compiler**

Language Translators

□ There are several kinds of programming language translators

- Assemblers

- perform one-to-one mapping from assembly code to machine code

- Compilers

- perform translation from *a high-level (machine-independent) statement* to an equivalent *short sequence of machine codes*

- Interpreters

- perform translation and execution of high-level statements at the same time; note that there is no intermediate machine code being generated

Formal vs Natural Languages

- ❑ Programming languages are “formal languages” since they are artificial languages defined precisely by grammars
- ❑ Natural (human) languages are not precisely defined by grammars, instead, grammars are created afterwards to “summarize” the language usage
 - Esperanto is an “formal” human language artificially developed in late 1870s.

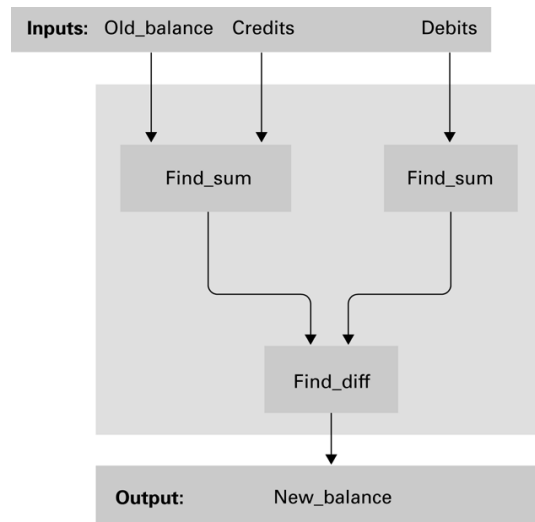
Programming Paradigms

- **Imperative (procedural):**

- A program is a sequence of commands
- Earliest way of programming

- **Functional:**

- A program is a description of a data flow (connections of functional units)
- Example, function for checking the balance in bank account:



A program in LISP programming language:

```
(diff (sum old_balance credits) (sum debits))
```

```
Total_credits = sum of all Credits  
Temp_balance = Old_balance + Total_credits  
Total_debits = sum of all Debits  
Balance = Temp_balance - Total_debits
```

Some Examples in Haskell

```
sumList :: [Integer] -> Integer
sumList [] = 0
sumList (x:xs) = x + sumList xs
```

```
factorial :: Integer -> Integer
factorial n = product [1..n]
```

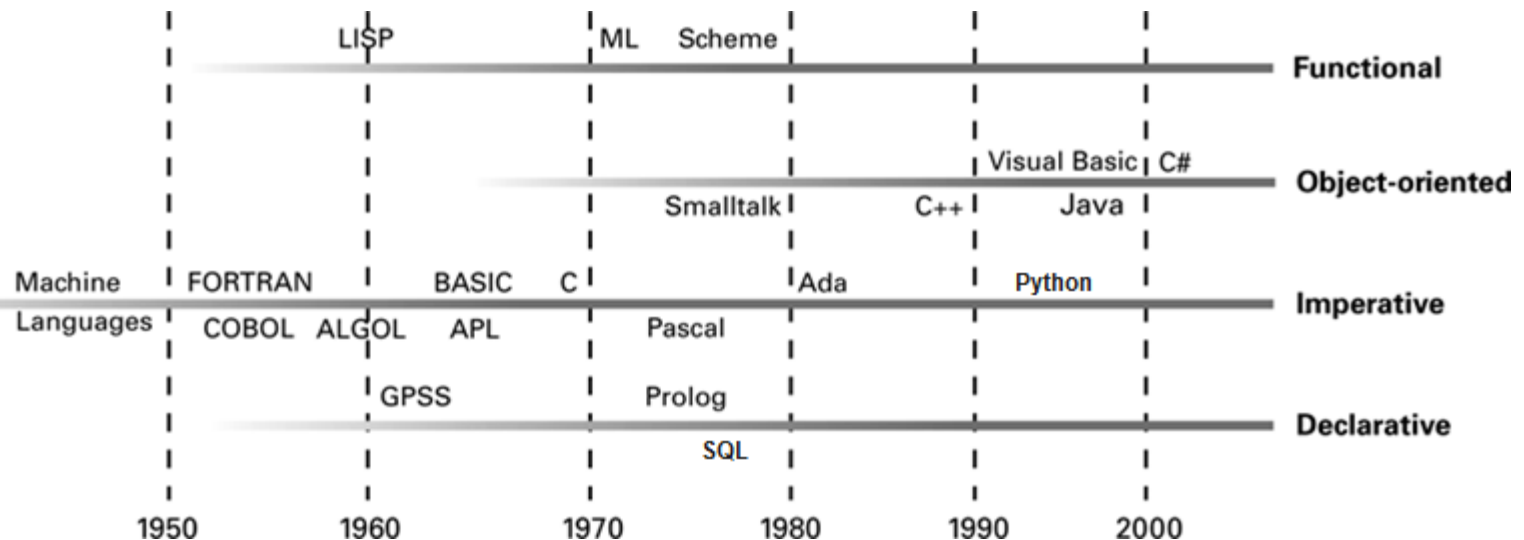
```
fib :: Integer -> Integer
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

```
gcd :: Integer -> Integer -> Integer
gcd 0 b = b
gcd a b = gcd (b `mod` a) a
```

Programming Paradigms

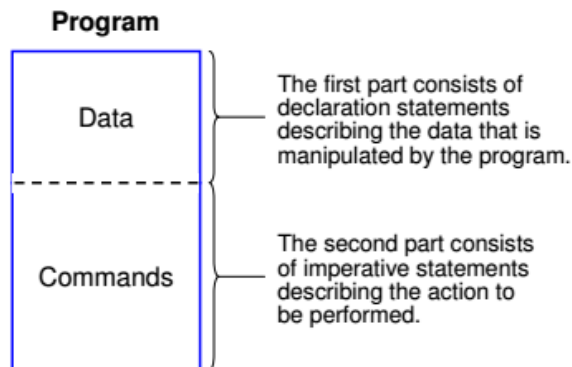
- **Declarative:**
 - Describes conditions that satisfy the intended solution; the specific steps needed to arrive at that solution are decided by the language itself.
 - Only works for a specific domain of problems
- **Object-oriented:**
 - Data-centric language which allows the creation of abstractions relevant to the problem setting
 - Functions (methods) are part of data they work with
 - Program is a specification for objects on working together to solve the problem

Evolution of Programming Paradigms



Imperative Programming Language

- ❑ An imperative programming language provides statements to:
 - Express constants and literals
 - Assign values to variables
 - Control the execution sequence of the program
 - Conditional control
 - Looping control
 - Commenting the program
 - Call procedural units
- ❑ The imperative programming paradigm is the most intuitive and effective way of expressing our commands to computers



Variable Declaration

- ❑ Variable (data) declarations in C, C++, C#, and Java are as follows:

- Scalar data declaration:

```
float Length, Width;  
int Price, Tax, Total;  
char Symbol;
```

- Aggregate data declarations:

```
int Scores[2][9];
```

⇒ array

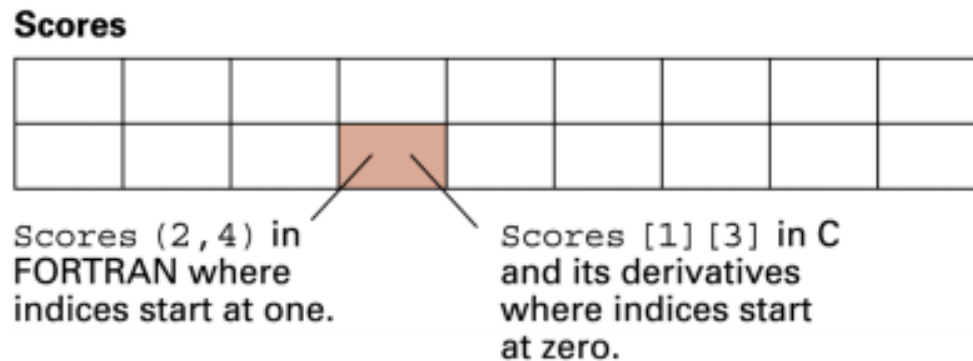
```
Struct {  
    char Name[8];  
    int Age;  
    float SkillRating;  
} Employee;
```

⇒ structure (heterogeneous array)

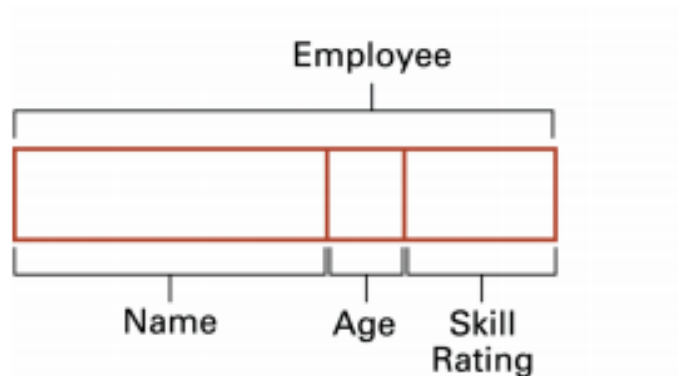
14/40

Memory Layout of Aggregate Data

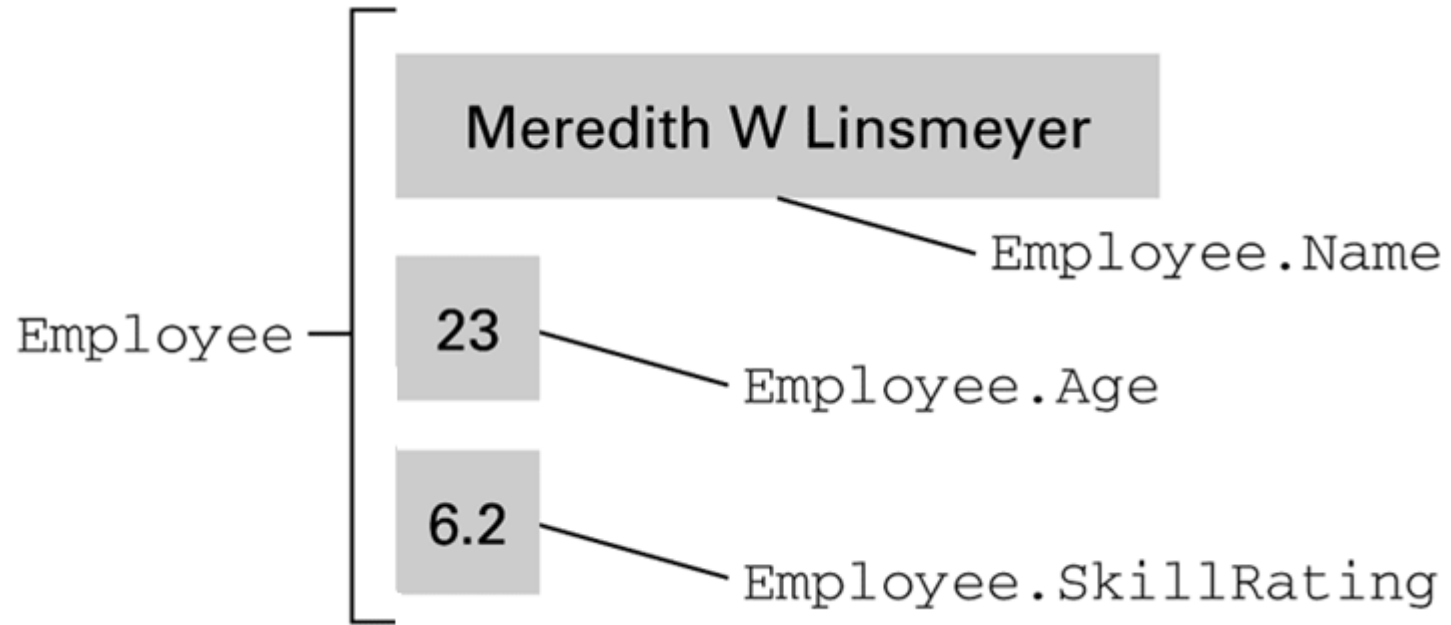
- ❑ A two-dimensional array with two rows and nine columns:



- ❑ A structure:



The conceptual structure of the aggregate type Employee

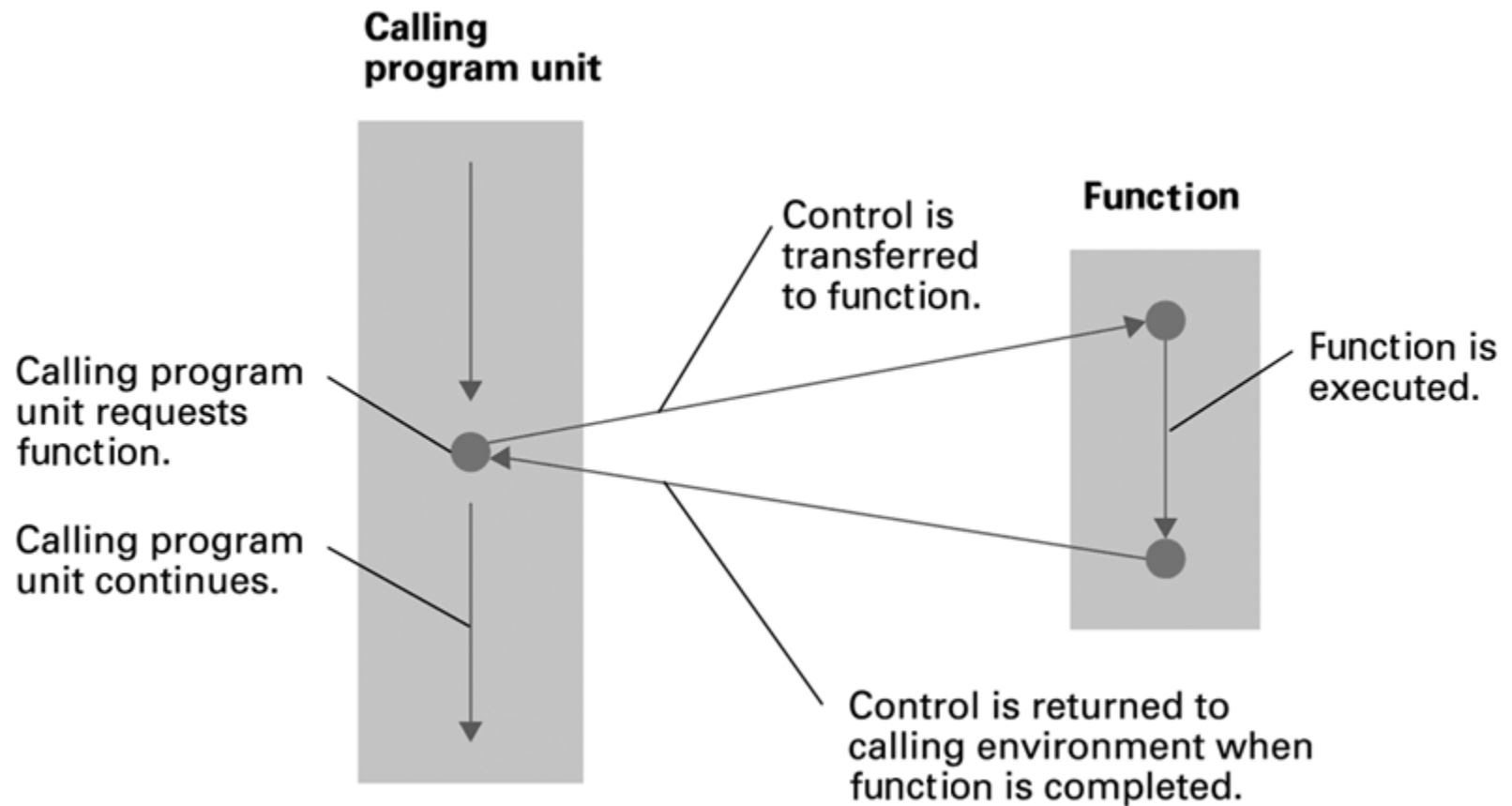


```
struct {    char  Name[25];  
          int   Age;  
          float SkillRating;  
} Employee;
```

Procedural Units

- Many terms for this concept:
 - Subprogram, subroutine, procedure, method, function
- Unit begins with the function's **header**
- Local versus Global Variables
- Formal versus Actual Parameters
- Passing parameters by value versus reference

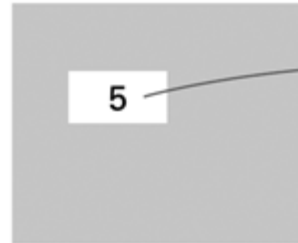
The flow of control involving a function



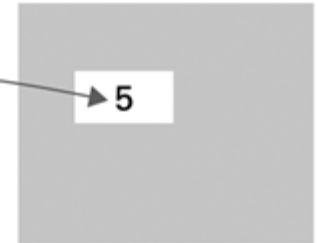
Executing the function and passing parameters by value

-
- a.** When the function is called, a copy of the data is given to the function

Calling environment



Function's environment



-
- b.** and the function manipulates its copy.

Calling environment

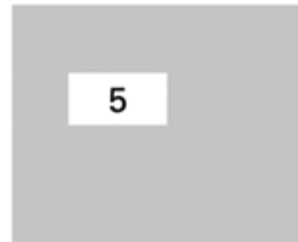


Function's environment



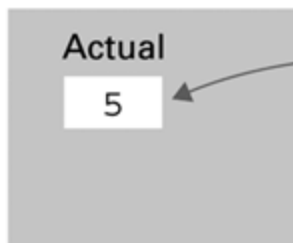
-
- c.** Thus, when the function has terminated, the calling environment has not been changed.

Calling environment

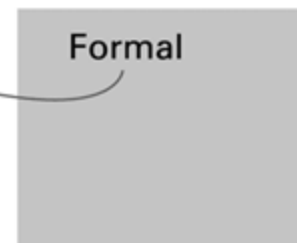


-
- a. When the function is called, the formal parameter becomes a reference to the actual parameter.

Calling environment

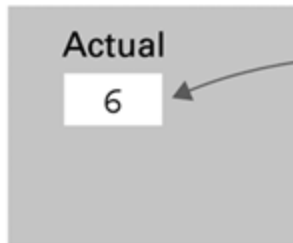


Function's environment :

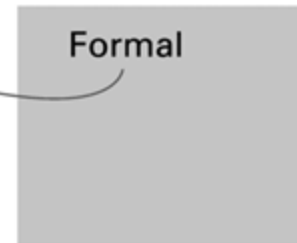


-
- b. Thus, changes directed by the function are made to the actual parameter

Calling environment

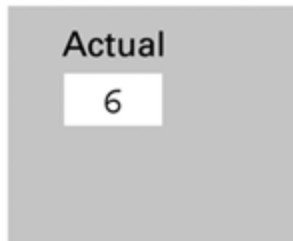


Function's environment



-
- c. and are, therefore, preserved after the function has terminated.

Calling environment

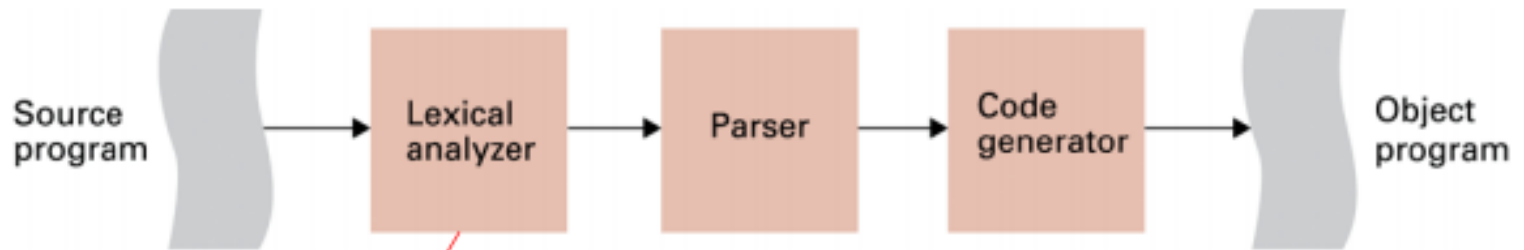


Executing the function
and passing parameters
by reference

LANGUAGE IMPLEMENTATION

Translating Program into Executable

- ❑ A compiler translates a program into machine codes via the following steps:



Lexical analyzer converts alpha-numerical symbols in the source program to tokens; for example, if each token is specified by a 16-bit number, a lexical analyzer may perform the following conversion:

`position = x_coord + y_coord * 7` → 0003 1001 0001 1002 0002 1003 2001

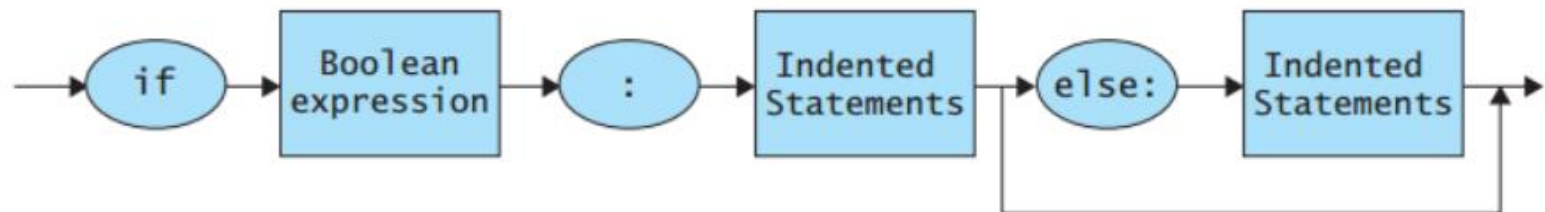
The first byte specifies the type of token,
0 – variables, 1 – operators, 2 – constants

The remaining bytes compose an
index to the token value tables

variables and their corresponding memory addresses are stored in a special table called ***symbol table***.

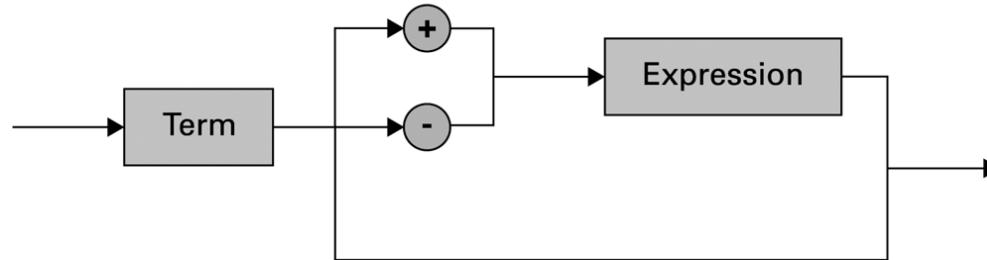
Syntax Diagram

- ❑ The parsing process is based on a set of rules that define the syntax of the programming language
 - The rules are called grammar
 - The rules can be expressed by syntax diagrams
- ❑ A syntax diagram of the “if-then-else” statement is as follows:

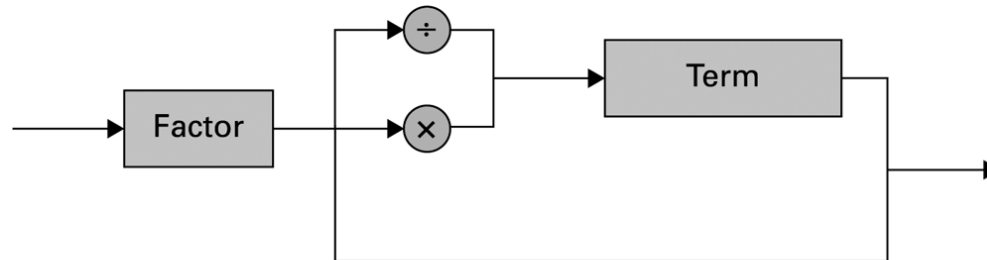


Syntax diagrams describing the structure of a simple algebraic expression

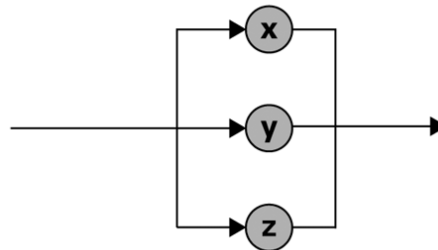
Expression



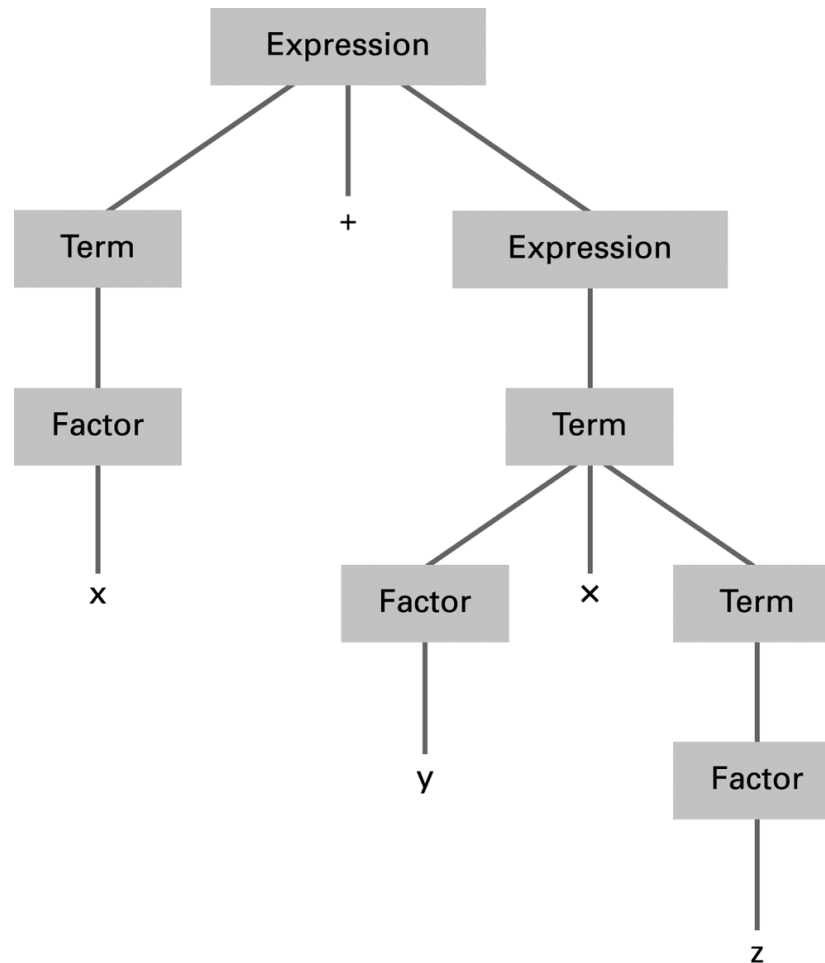
Term



Factor



The parse tree for the string $x + y * z$ based on the syntax



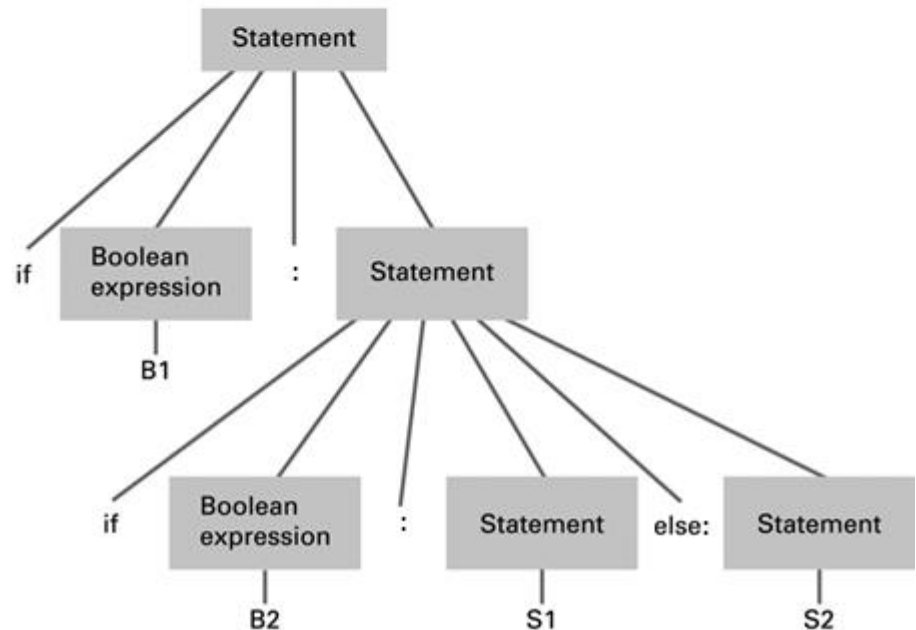
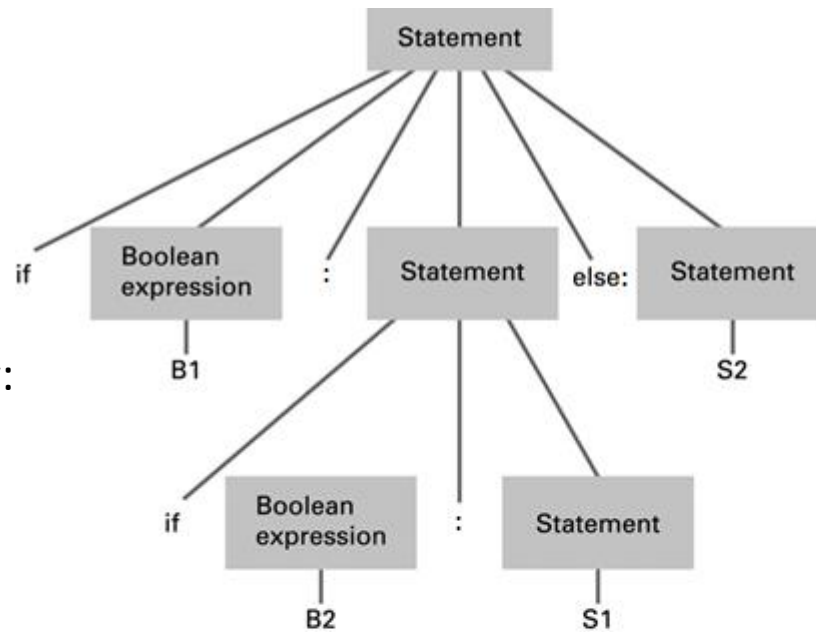
Two distinct parse trees for the statement:

```
if B1: if B2: S1 else: S2
```

Indentation can remove ambiguity:

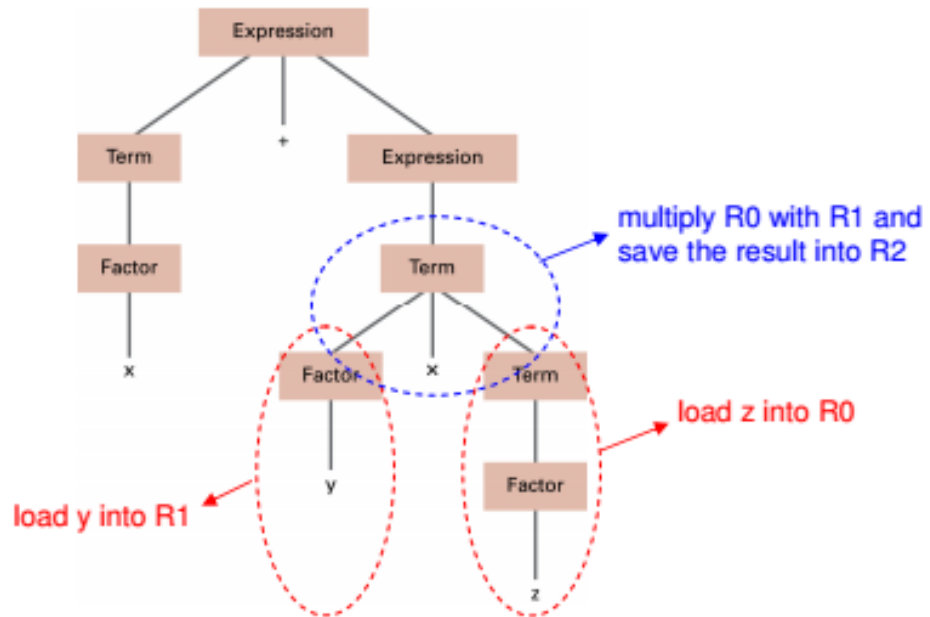
```
1) if B1:
    if B2:
        S1
    else:
        S2
```

```
1) if B1:
    if B2:
        S1
    else:
        S2
```



Code Generation and Optimization

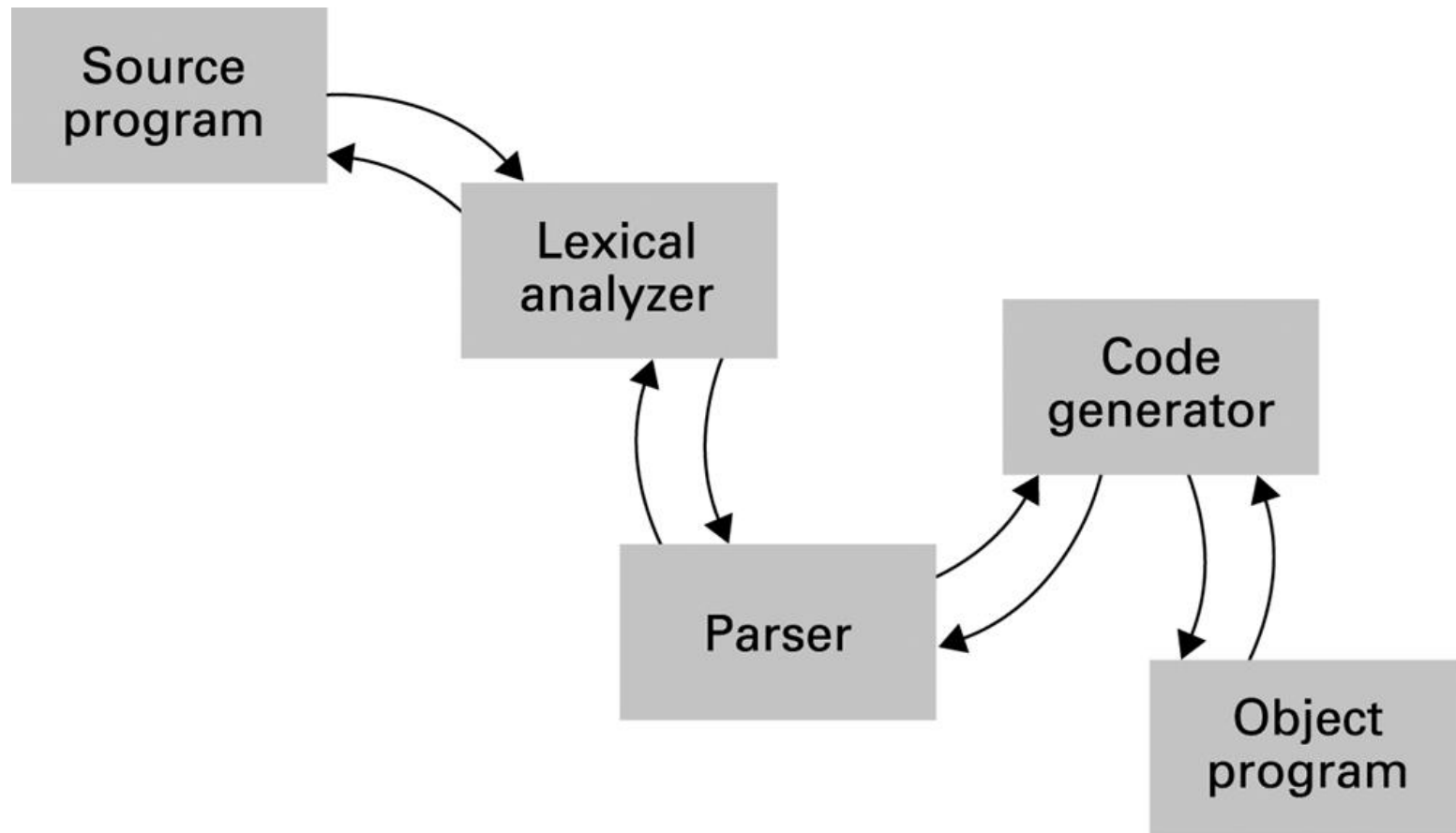
- Once the parse tree is done, one must generate machine codes for each sub-tree or node, for example, in bottom-up manner



- Code optimization is a technique for finding the best way to generate codes

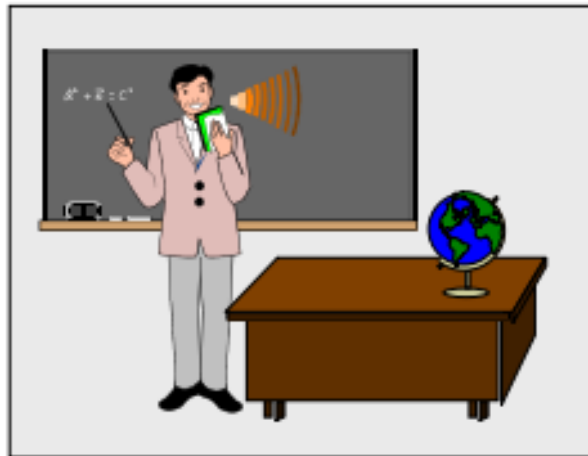
$x = y + z$
 $w = x + z$

An object-oriented approach to the translation process

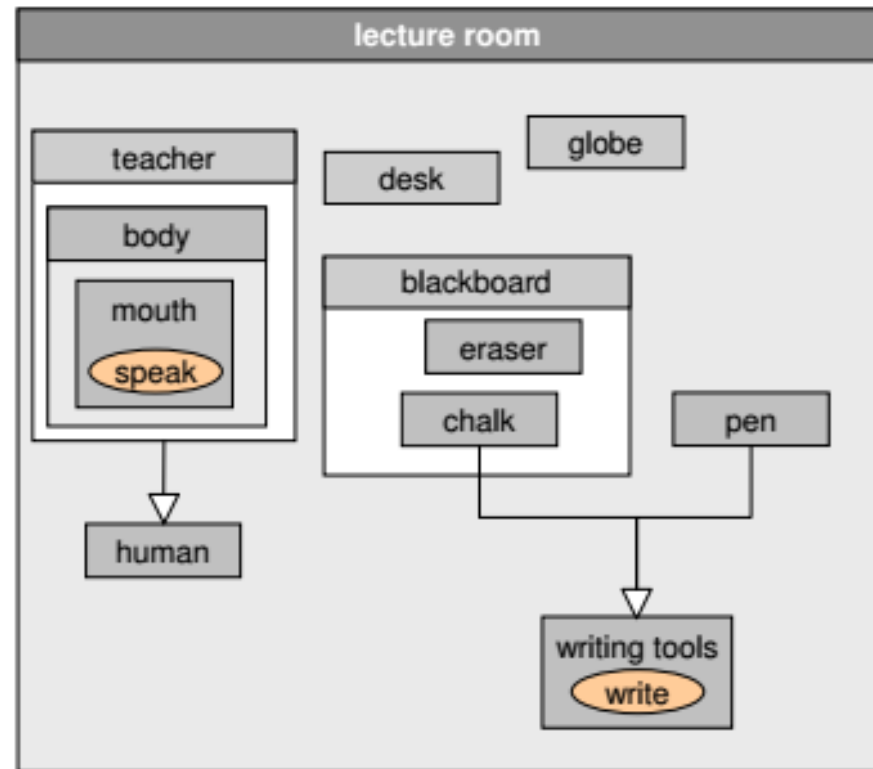


OTHER PROGRAMMING PARADIGMS

Object Oriented View of the World



An object-oriented
description of the lecturing process



32/40

Objects and Classes

- An object-oriented (OO) programming composed of a hierarchical structure of objects:
 - **Object:** Active program unit containing both data and procedures
 - **Class:** A template from which objects are constructed
- An object is called an **instance** of the class.
- An OO program is composed of the declaration of different types (classes) and how the instances of these classes created and interact together.

The structure of a class describing a laser weapon in a computer game

```
class LaserClass
```

```
{  int RemainingPower = 100;
```

```
    void turnRight ( )  
    { ... }
```

```
    void turnLeft ( )  
    { ... }
```

```
    void fire ( )  
    { ... }
```

```
}
```

Description of the data
that will reside inside of
each object of this "type."

Methods describing how an
object of this "type" should
respond to various messages

Components of an Object

- **Instance Variable:** Variable within an object
 - Holds information within the object
- **Method:** Procedure within an object
 - Describes the actions that the object can perform
- **Constructor:** Special method used to initialize a new object when it is first constructed

A class with a constructor

```
class LaserClass  
{ int RemainingPower;
```

Constructor assigns a
value to RemainingPower
when an object is created.

```
LaserClass (InitialPower)  
{ RemainingPower = InitialPower;  
}
```

```
void turnRight ( )  
{ ... }
```

```
void turnLeft ( )  
{ ... }
```

```
void fire ( )  
{ ... }
```

```
}
```

Object Integrity

- **Encapsulation:** A way of restricting access to the internal components of an object
 - Private
 - Public

Components in the class are designated public or private depending on whether they should be accessible from other program units.

```
class LaserClass
{private int RemainingPower;

public LaserClass (InitialPower)
{RemainingPower = InitialPower;
}

public void turnRight ( )
{ ... }

public void turnLeft ( )
{ ... }

public void fire ( )
{ ... }
}
```

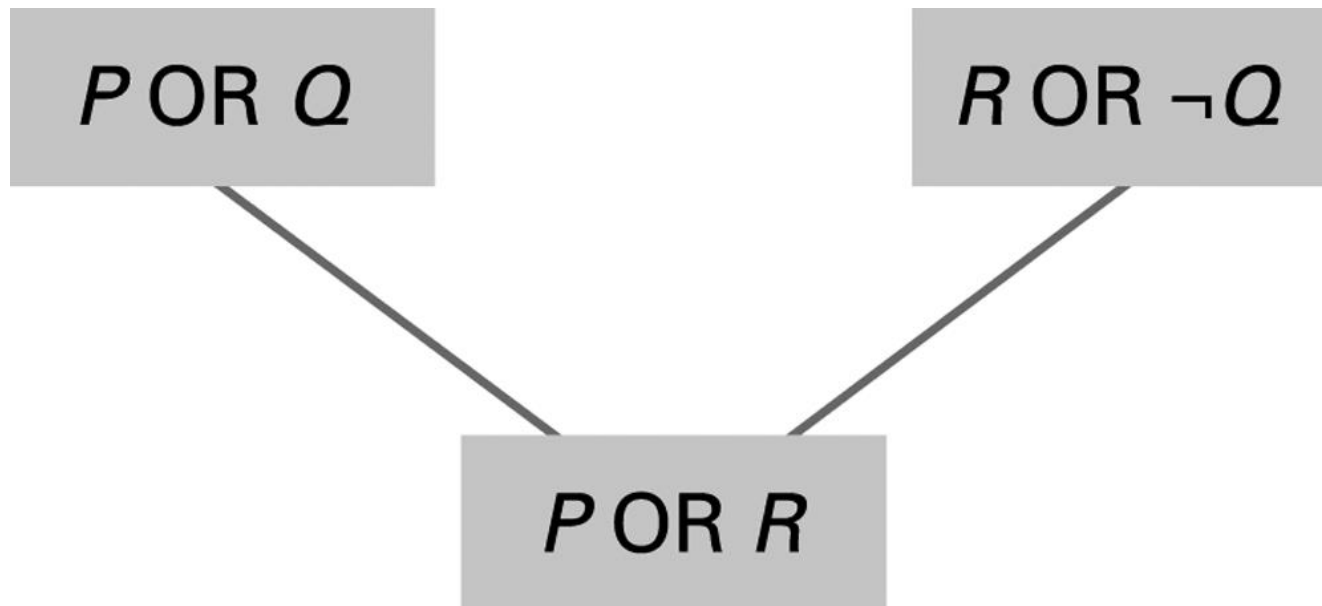
Additional Object-oriented Concepts

- **Inheritance:** Allows new classes to be defined in terms of previously defined classes
- **Polymorphism:** Allows method calls to be interpreted by the object that receives the call

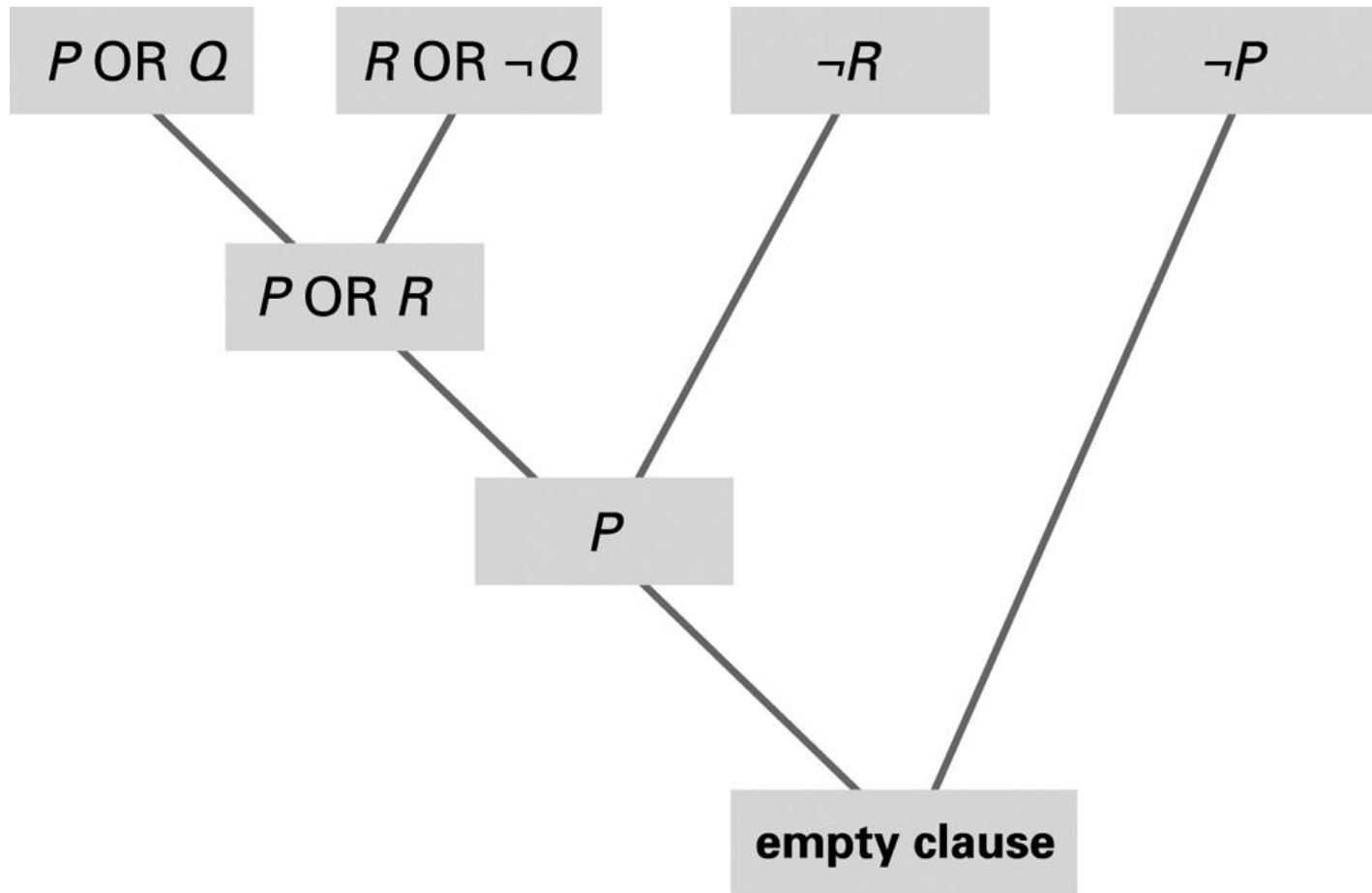
Declarative Programming

- **Resolution:** Combining two or more statements to produce a new statement (that is a logical consequence of the originals).
 - Example: $(P \text{ OR } Q) \text{ AND } (R \text{ OR } \neg Q)$ resolves to $(P \text{ OR } R)$
 - **Resolvent:** A new statement deduced by resolution
 - **Clause form:** A statement whose elementary components are connected by the Boolean operation OR
 - **Conjunctive normal form (CNF):** A statement whose elementary components are in clause form and connected by Boolean operation AND
- **Unification:** Assigning a value to a variable so that two statements become “compatible.”

Resolving the statements $(P \text{ OR } Q)$ and $(R \text{ OR } \neg Q)$
to produce $(P \text{ OR } R)$



Resolving the statements $(P \text{ OR } Q)$, $(R \text{ OR } \neg Q)$, $\neg R$, and $\neg P$



Prolog

- **Fact:** A Prolog statement establishing a fact
 - Consists of a single predicate
 - Form: *predicateName(arguments)*.
 - Example: `parent(bill, mary).`
- **Rule:** A Prolog statement establishing a general rule
 - Form: *conclusion :- premise.*
 - `:-` means “if”
 - Example: `wise(X) :- old(X).`
 - Example: `faster(X,Z) :- faster(X,Y), faster(Y,Z).`
- **Try using similar tool online:**
 - <https://logictools.org>
 - OR install [SWI-Prolog](#) on your computer.

Overview of Programs in Different Programming Paradigms

- **Imperative paradigm:** Program is a sequence of instructions to run
- **Declarative paradigm:** Program is a problem statement, formal description of the problem to be solved or accomplished.
- **Functional paradigm:** Program is a data flow from one functional unit to another functional unit avoiding the need for maintaining state information.
- **Object-oriented paradigm:** Program is a specification of abstractions in the form of classes and a specification on creation of their instances and their mutual interactions.

Suggested Reading/Activity

- How computers read code:
 - <https://www.youtube.com/watch?v=QXjU9qTsYCc>
- Why functional programming matters?
 - <https://www.youtube.com/watch?v=XrNdvWqxBvA>
- **Learn Haskell** – a purely functional programming language
 - Install the compiler: <https://www.haskell.org/ghcup/>
 - Tutorial: <http://learnyouahaskell.com/starting-out>
- **Learn Prolog** – a declarative programming language for logic programming.
 - Install the compiler: <http://www.swi-prolog.org/>
 - Tutorials:
 - <https://www.swi-prolog.org/pldoc/man?section=quickstart>
 - <https://book.simply-logical.space/src/simply-logical.html>